

Military Alert System Report

Prepared by: Minh Tue Trinh - a1902421

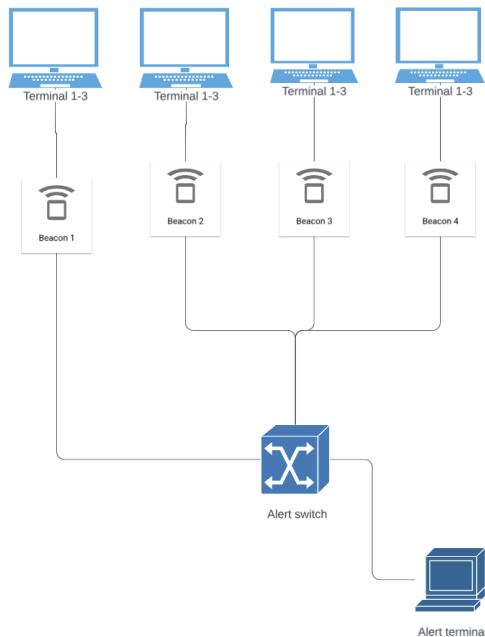
Course: Network Design

Attached files:

- mininet_topology_builder.py: build the network topology of the alert system
- military_alert_system.py: functionality implementation
- a1902421-Military Alert System Report
- A supporting demo video showing my dedication to the assignment. Demonstrating my work and approach to the assignment: [Network Design - Assignment 3](#)

1. Network topology

Minh Tue Trinh - 1902421



The topology follows the provided Military Alert System, where there is one Alert Terminal, one Central Alert Switch, four Beacon Switches and three terminals connected to each beacon switch.

Device	Purpose
Alert terminal	Sending alert messages using IP address and UDP ports
Alert switch	This switch will be connected to the alert terminal. It is responsible for sending traffic to the SDN controller
Beacon switch 1	Connects to Beacon 1 terminal
Beacon switch 2	Connects to Beacon 2 terminal
Beacon switch 3	Connects to Beacon 3 terminal
Beacon switch 4	Connects to Beacon 4 terminal
Beacon terminals	Receiving rewritten UDP alert messages

The main function inside `mininet_topology_builder.py` that builds the topology for this assignment is `militaryAlertSystem()`, and registered as `topos` in the topology dictionary under `MAS`.

```
mininet_topology_builder.py > ...
27 def militaryAlertSystem():
28     net = createInitialNetwork()
29     alert = net.addHost('alert', ip='10.0.0.1/24', mac='00:00:00:00:aa:01')
30     asw = net.addSwitch('asw', dpid='1', protocols='OpenFlow13')
31     beacons = {
32         'b1': net.addSwitch('b1', dpid='2', protocols='OpenFlow13'),
33         'b2': net.addSwitch('b2', dpid='3', protocols='OpenFlow13'),
34         'b3': net.addSwitch('b3', dpid='4', protocols='OpenFlow13'),
35         'b4': net.addSwitch('b4', dpid='5', protocols='OpenFlow13'),
36     }
37
38     terminals = {}
39     for beacon_count in range(1,5):
40         for terminal_count in range(1,4):
41             name = 'b%d\t%d' % (beacon_count, terminal_count)
42             terminals[name] = net.addHost(name, ip='10.0.%d.%d/24' % (beacon_count, terminal_count), mac='00:00:00:00:
43
44     net.addLink(alert,asw)
45     net.addLink(asw, beacons['b1'])
46     net.addLink(asw, beacons['b2'])
47     net.addLink(asw, beacons['b3'])
48     net.addLink(asw, beacons['b4'])
49
50     for beacon_number in range(1,5):
51         beacon = 'b%d' % beacon_number
52         for terminal_number in range(1,4):
53             terminal_name = 'b%d\t%d' % (beacon_number, terminal_number)
54             net.addLink(beacons[beacon], terminals[terminal_name])
55
56     net.start()
57     alert.cmd('arp -s 10.0.0.254 00:00:00:00:fe:fe')
58     alert.cmd('ip neigh replace 10.0.0.254 lladdr 00:00:00:00:fe:fe nud permanent dev alert-eth0')
59     alert.cmd('ip route replace default via 10.0.0.254 dev alert-eth0')
60
61     print("\nMilitary alert system topology started. Host default gateway can be found at: 10.0.0.254. Fake gateway MA
62     print("Given the topology: Alert -> ASW -> Beacons (b1, b2, b3, b4) -> Terminals (t1, t2, t3)")
63
64     safeMininetStartupAndExit(net)
65
66     topos = {'MAS': (lambda: militaryAlertSystem())}
```

In order to run the topology, type in the terminal:
`sudo mn --custom mininet_topology_builder.py --topo MAS`

```

student@sdn-dev-env:/opt/workspace/military_system_alert$ sudo mn --custom mininet_topology_builder
.py --topo MAS
Do you want to connect to an SDN Controller? Y/N: y
Controller is reachable, adding it now.
*** Configuring hosts
alert b1t1 b1t2 b1t3 b2t1 b2t2 b2t3 b3t1 b3t2 b3t3 b4t1 b4t2 b4t3
*** Starting controller
c0
*** Starting 5 switches
asw b1 b2 b3 b4 ...
Military alert system topology started. Host default gateway can be found at: 10.0.0.254. Fake gate
way MAC address is 00:00:00:00:fe:fe
Given the topology: Alert -> ASW -> Beacons (b1, b2, b3, b4) -> Terminals (t1, t2, t3)
*** Starting CLI:
mininet> links
alert-eth0->asw-eth1 (OK OK)
asw-eth2->b1-eth1 (OK OK)
asw-eth3->b2-eth1 (OK OK)
asw-eth4->b3-eth1 (OK OK)
asw-eth5->b4-eth1 (OK OK)
b1-eth2->b1t1-eth0 (OK OK)
b1-eth3->b1t2-eth0 (OK OK)
b1-eth4->b1t3-eth0 (OK OK)
b2-eth2->b2t1-eth0 (OK OK)
b2-eth3->b2t2-eth0 (OK OK)
b2-eth4->b2t3-eth0 (OK OK)
b3-eth2->b3t1-eth0 (OK OK)
b3-eth3->b3t2-eth0 (OK OK)
b3-eth4->b3t3-eth0 (OK OK)
b4-eth2->b4t1-eth0 (OK OK)
b4-eth3->b4t2-eth0 (OK OK)
b4-eth4->b4t3-eth0 (OK OK)
mininet> alert ip route
default via 10.0.0.254 dev alert-eth0
10.0.0.0/24 dev alert-eth0 proto kernel scope link src 10.0.0.1
mininet> alert ping -c 1 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
--- 1.1.1.1 ping statistics ---
1 packets transmitted, 0 received, 100% packet loss, time 0ms
mininet> THIS IS DONE BY MINH TUE TRINH - a1902421[]

```

Note that the Ryu-based function must be initialised in another terminal before starting the topology builder. Otherwise, the program will crash and result in an error:

```

student@sdn-dev-env:/opt/workspace/military_system_alert$ ryu-manager military_alert_system.py
loading app military_alert_system.py
loading app ryu.controller.ofp_handler
instantiating app military_alert_system.py of MilitaryAlertSystem
instantiating app ryu.controller.ofp_handler of OFPHandler

```

The alert switch is given datapathID 1, while the beacon switches use datapathID from 2 to 5. The IDs are fixed these are very important since the Ryu controller uses them to identify which switch is connected. The alert terminal is configured with a fake default gateway of 10.0.0.254 and the fake gateway MAC address of 00:00:00:00:fe:fe

```

Military alert system topology started. Host default gateway can be found at: 10.0.0.254. Fake gate
way MAC address is 00:00:00:00:fe:fe

```

This gateway does not exist, as the program only uses it so that the alert terminal will try to forward UDP packets to a fake destination IP such as 192.168.1.1 or 8.8.8.8. A static ARP entry is added to the fake gateway so the host does not get stuck performing ARP before sending packets

```

#packet protocol parsers allow cotrolller to inspect Ethernet, ipv4, dup and arp header in the packet message
from ryu.lib.packet import arp, ethernet, ether_types, ipv4, packet, udp

#drop arp. the alert terminal already has a fake arp entry
if eth_frame is not None and eth_frame.ethertype == ether_types.ETH_TYPE_ARP:
    arp_packet = packet_data.get_protocol(arp.arp)
    target = arp_packet.dst_ip if arp_packet is not None else "unknown"
    self._log_drop("unexpected ARP for %s" % target)
    return

```

The generated topology from the mininet_topology_builder.py is:

```

Given the topology: Alert -> ASW -> Beacons (b1, b2, b3, b4) -> Terminals (t1, t2, t3)

```

2. SDN Controller

The implemented Ryu SDN serves as the traffic controller for the MAS, determining the logic of network traffic flow. Ryu is imported from the virtual environment and applied inside `military_alert_system.py`:

```
military_alert_system.py > ...
1  from ryu.base import app_manager #ryu base class
2
3  #ryu controller is responsible for implementing logics to handle switch events (connecting switch, sending packets, openflow management)
4  from ryu.controller import ofp_event
5  from ryu.controller.handler import CONFIG_DISPATCHER, MAIN_DISPATCHER, set_ev_cls
6
7  #packet protocol parsers allow controller to inspect Ethernet, ipv4, dup and arp header in the packet message
8  from ryu.lib.packet import arp, ethernet, ether_types, ipv4, packet, udp
9  from ryu.ofproto import inet, ofproto_v1_3
```

The controller is responsible for:

- Receiving packets from the Alert Switch
- Inspecting UDP packets from the Alert Terminal
- Reading the destination IP addresses
- Reading the UDP destination port
- Mapping the IP code to the correct beacon switch
- Mapping the port code to the terminal
- Dropping unknown switches and traffic
- Rewriting packets
- Logging

The controller rewrites valid alert packets so that:

Source IP address becomes: 254.254.254.254

Destination IP address: 253.253.253.253

Destination UDP port: 1

The destination UDP port is changed to 1 so that receiving terminals can listen using `tcpdump udp port 1 -A`

switch_features_handler(): The function will run when the switch first connects to Ryu. It gets the datapathid as the parameter and checks the conditions to see whether the switch is an alert switch or a beacon switch. After determining the switch type, the function will begin configuring the switch using `_configure_alert_switch()`, which will be explained later. If the `switch_id` does not match either the alert switch or one of the beacon switches, the function will drop that unknown switch.

```
#first runs when each switch first connects to ryu
def switch_features_handler(self, ev):
    datapath = ev.msg.datapath
    switch_id = datapath.id
    parser = datapath.ofproto_parser
    ofproto = datapath.ofproto

    self.datapaths[switch_id] = datapath
    self.logger.info("[MAS] Switch connected: DPID %s", switch_id)
    #configure the central alert switch
    if switch_id == ALERT_SWITCH_DATAPATHID:
        self._configure_alert_switch(datapath, parser, ofproto)
    #configure beacon switches
    elif switch_id in BEACON_BY_DATAPATHID:
        self._configure_beacon_switch(datapath, parser)
    else:
        self.logger.warning("[MAS] Unknown switch DPID %s; installing drop", switch_id) #unknown switches
        self.install_flow(datapath, priority=0, match=parser.OFPMatch(), actions=[])
```

_configure_beacon_switch(): The function is used for setting up the central alert switch when it first links to Ryu. The alert switch is the most important part in the MAS design as all the coded alert traffic from the alert terminal must go through it first before forwarding any further. The function will set a general rule that sends traffic from the alert terminal port to SDN controller. Meaning, instead of setting static flow for each IP address and UDP port, it will generally send the packet to the controller. This is crucial and meet the expectation where the secret path will be hidden with the Ryu controller rather than being exposed in the alert switch flow mapping.

```
def _configure_alert_switch(self, datapath, parser, ofproto):
    controller_action = [
        parser.OFPACTIONOutput(
            ofproto.OFPP_CONTROLLER,
            ofproto.OFPCML_NO_BUFFER
        )
    ]

    self.install_flow(
        datapath,
        priority=100,
        match=parser.OFPMatch(in_port=ALERT_TERMINAL_PORT),
        actions=controller_action
    )
    self.install_flow(datapath, priority=0, match=parser.OFPMatch(), actions=[])

    self.logger.info(
        "[MAS] Alert switch configured: port %s traffic goes to controller",
        ALERT_TERMINAL_PORT
    )
```

For example, the alert switch does not need to know that 192.168.1.1 = Beacon 1, the packet will be sent to the Ryu controller for further processing.

_configure_beacon_switch(): The function's name is pretty much self-explanatory where it will configure the Beacon switches, in this scenario, it will be 4 Beacon switches. Those switches do not have to decode IP addresses and UDP ports as the controller already did that before reaching the beacons. Their purpose is to match packets with rewritten values of the source IP, destination IP and UDP destination port. The flow checks the MAC address to determine the destination terminal. Due to the fact that the original UDP port code is rewritten to 1 so the beacon switch can not decide the terminal using the original port code. To address this, Ryu controller will set the destination MAC address before sending the packet, and the beacon switches rely on this to send to the correct terminal port. In short, the function only forwards the rewritten packets to the correct terminal ports.

```
def _configure_beacon_switch(self, datapath, parser):
    #the alert switch should not store secret routing logic
    #sending traffic from the alert terminal to the controller
    beacon = BEACON_BY_DATAPATHID[datapath.id]

    for terminal, out_port in BEACON_TERMINAL_PORT_TABLE.items():
        match = parser.OFPMatch(
            in_port=1,
            eth_type=ether_types.ETH_TYPE_IP,
            ip_proto=inet.IPPROTO_UDP,
            ipv4_src=REWRITE_SRC_IP,
            ipv4_dst=REWRITE_DST_IP,
            udp_dst=REWRITE_UDP_PORT,
            eth_dst=TERMINAL_MAC_ADDRESS[beacon][terminal]
        )
        actions = [parser.OFPACTIONOutput(out_port)]
        self.install_flow(datapath, priority=100, match=match, actions=actions)
    #send packets entering from the alert terminal port to controller
    self.install_flow(datapath, priority=0, match=parser.OFPMatch(), actions=[])#drop everything else by default
    self.logger.info("[MAS] %s forwarding rules installed", BEACON_LABELS[beacon])
```

packet_in_handler(): This is a provided function from the template, and I have successfully developed further to match the requirements of the MAS design. This is the main decision-making function in the controller. Whenever the alert switch sends the PacketIn events in Ryu, packet_in_handler() will be triggered. Firstly, it checks which switch sends the packet, and any packet that does not come from the alert switch will be dropped, then checks which port the packet enters from. Any packets sent from the alert terminal port will be treated as alerts!!! So that any valid traffic will not be triggered as alerts, keeping the traffic flow safe and consistent.

```
#only the alert switch should make the MAS decisions
if switch_id != ALERT_SWITCH_DATAPATHID:
    self._log_drop("PacketIn from non-alert switch DPID %s" % switch_id)
    return
#only accept packets from the alert terminal path
if in_port != ALERT_TERMINAL_PORT:
    self._log_drop("traffic entered alert switch from unexpected port %s" % in_port)
    return
```

Then the function parses the packet header to get the Ethernet frame, IPv4 and UDP packets. Any non-IPv4, non-UDP will be ignored as MAS only forwards valid UDP alert packets. So if the

alert switch sends a ping, the function will treat it as ICMP rather than UDP so that the controller logs it as non-UDP traffic and does not forward it to any terminal

```
#if not ipv4 --> drop
if ip_v4 is None:
    self._log_drop("non-IPv4 packet")
    return

#if not udp --> drop
if ip_v4.proto != inet.IPPROTO_UDP or udp_packet is None:
    self._log_drop(
        "non-UDP IPv4 packet to %s protocol=%s" % (ip_v4.dst, ip_v4.proto)
    )
    return

dst_ip = ip_v4.dst
dst_port = udp_packet.dst_port

self.logger.info(
    "[MAS] Received UDP alert candidate: dst_ip=%s dst_port=%s",
    dst_ip,
    dst_port
)
```

The controller will get the `dst_ip` and `dst_port` which are IPv4 address and UDP port respectively. They will be treated as codes rather than network addresses. `dst_ip` chooses beacon target while the `dst_port` chooses the terminal target. Those chosen values will then be matched with the mapping table. If the `dst_ip` and the `dst_port` do not match the values in the mapping table then they will be dropped.

```
#check fake beacon code
if dst_ip not in BEACON_IP_TABLE:
    self._log_drop("invalid beacon IP code %s" % dst_ip)
    return

#check fake terminal code
if dst_port not in TERMINAL_PORT_TABLE:
    self._log_drop("invalid terminal UDP port code %s" % dst_port)
    return
```

If both the `dst_ip` and `dst_port` pass, the function will begin to decode them. Then controller can forward a rewritten copy of the packet for each selected beacon and terminal combination.


```

payload = self._extract_payload(packet_data)
if payload:
    self.logger.info("[MAS] Payload: %s", payload)

#log the rewrite action
self.logger.info(
    "[MAS] Rewriting src_ip=%s dst_ip=%s udp_dst=%s",
    REWRITE_SRC_IP,
    REWRITE_DST_IP,
    REWRITE_UDP_PORT
)
#forward the rewritten packets to the selected beacon or terminal
self.forward_alert(datapath, msg, target_beacons, target_terminals)

```

forward_alert(): performs the actual packet forwarding and rewriting. It will be triggered after `packet_in_handler()` has confirmed the valid packets and decoded targets (beacons and terminals). It sends the rewritten packet out of the correct alert switch port to the selected beacon. For instance, target: Beacon 1, the packet is sent from the alert switch port connected to b1

```

def forward_alert(self, datapath, msg, target_beacons, target_terminals):
    parser = datapath.ofproto_parser

    for beacon in target_beacons:
        alert_switch_port = ALERT_SWITCH_OUTPUT_PORT_TABLE[beacon]

        for terminal in target_terminals:
            terminal_mac_address = TERMINAL_MAC_ADDRESS[beacon][terminal]
            actions = [
                parser.OFPActionSetField(eth_src=FAKE_GATEWAY_MAC),
                parser.OFPActionSetField(eth_dst=terminal_mac_address),
                parser.OFPActionSetField(ipv4_src=REWRITE_SRC_IP),
                parser.OFPActionSetField(ipv4_dst=REWRITE_DST_IP),
                parser.OFPActionSetField(udp_dst=REWRITE_UDP_PORT),
                parser.OFPActionOutput(alert_switch_port),
            ]

            self.send_packet_out(
                datapath=datapath,
                in_port=msg.match["in_port"],
                actions=actions,
                data=msg.data
            )

            self.logger.info(
                "[MAS] Forwarding to %s / %s via alert switch port %s",
                BEACON_LABELS[beacon],
                TERMINAL_LABELS[terminal],
                alert_switch_port
            )

```

3. Logical flow

militaryAlertSystem() flow:

Start Mininet topology → Create alert terminal, alert switch, beacon switches (b1, b2, b3, b4), terminal hosts (b1t1 - b4t3) → Connect alert terminal to alert switch → connect alert switch to each beacon switch → connect each beacon to its three terminals → start the network → Add fake ARP entry and default gateway

switch_features_handler() flow:

Switch connects to Ryu → Controller reads the switch datapathid → store switch datapath → Is switch datapathid 1?

→ Yes: Alert switch

→ No: if datapathid is 2, 3, 4 or 5 → beacon switch; else: drop

_configure_alert_switch() flow:

Alert switch connects → Install rule for traffic for traffic from alert terminal port → send to SDN controller → Alert switch → packetin to Ryu controller

_configure_beacon_switch() flow:

Connecting beacon switches → determining Beacon using datapathid → For T1, 2, 3: install rule matching rewritten MAS packet, match source IP, destination IP, UDP destination, selected terminal MAC address and output to correct terminal port → drop rule → beacon switch is set

packet_in_handler() flow:

Alert switch sends packets → Controller receives packets and checks if it comes from alert switch and enters from the alert terminal port → parse ethernet, ipv4 and UDP headers → if not ipv4 and udp = drop/if it is ARP = drop → read the destination IP code → read udp destination port code → if not valid drop else: decode target beacons and terminals → read payload if possible → logs → call forward_alert()

forward_alert():

Receive valid decode alert:

→ For each target beacon:

Get alert switch output port for that specific beacon → for each target terminal:

Get Terminal MAC address → Rewrite ethernet destination MAC to selected terminal → rewrite source IP to 254.254.254.254 → rewrite destination IP to 253.253.253.253 → rewrite UDP destination port to 1 → Send packetout from alert switch to beacon → log action

For example:

192.168.1.1:22 → beacon 1 Terminal 2 → rewrite packet → Output from alert switch to b1 → b1 forwards to b1t2

The big picture of the MAS:

Alert Terminal → Fake IP/port coded UDP packet → Alert Switch → PacketIn controller → Validate packet → Decode beacon and terminal → rewrite packet → PacketOut to Beacon → Beacon switch forwards → Correct terminal receives on UDP port 1

4. Launching MAS

Open the SDN environment via VM and open localhost:81

In one terminal - Starts the SDN controller: `ryu-manager military_alert_system.py`

In another terminal - Launch mininet topology: `sudo mn --custom mininet_topology_builder.py --topo MAS`